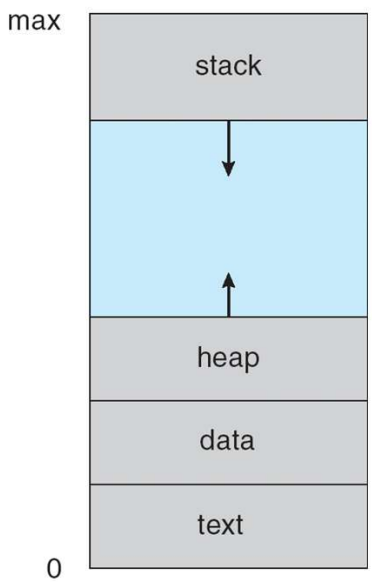
Chapter 3: Processes

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- Communication in Client Server Systems

Process Concept

- An operating system executes a variety of programs:
  - Batch system – **jobs**
  - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms ***job*** and ***process*** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion

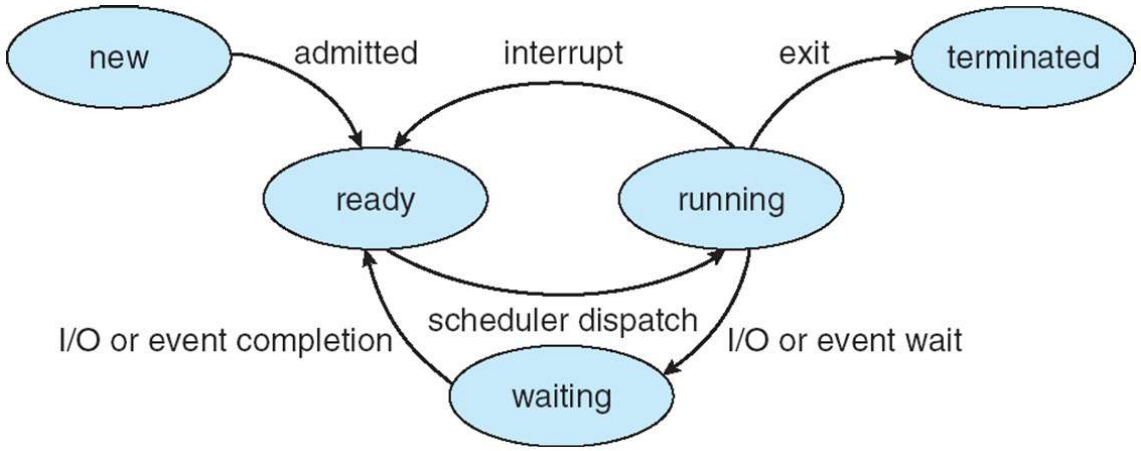


Process Concept

- Program is ***passive*** entity stored on disk (**executable file**), process is ***active***
  - Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
  - Consider multiple users executing the same program

Process State

- As a process executes, it changes **state**
  - **new**: The process is being created
  - **running**: Instructions are being executed
  - **waiting**: The process is waiting for some event to occur
  - **ready**: The process is waiting to be assigned to a processor
  - **terminated**: The process has finished execution



  - Only one process can be running on any processor at any instant. Many processes may be ready or waiting, however.

Process Control Block (PCB)

Information associated with each process

(also called **task control block**)

- Process state – running, waiting, etc
- Program counter – location of instruction to next execute
- CPU registers – contents of all process-centric registers
- CPU scheduling information- priorities, scheduling queue pointers
- Memory-management information – memory allocated to the process
- Accounting information – CPU used, clock time elapsed since start, time limits
- I/O status information – I/O devices allocated to process, list of open files

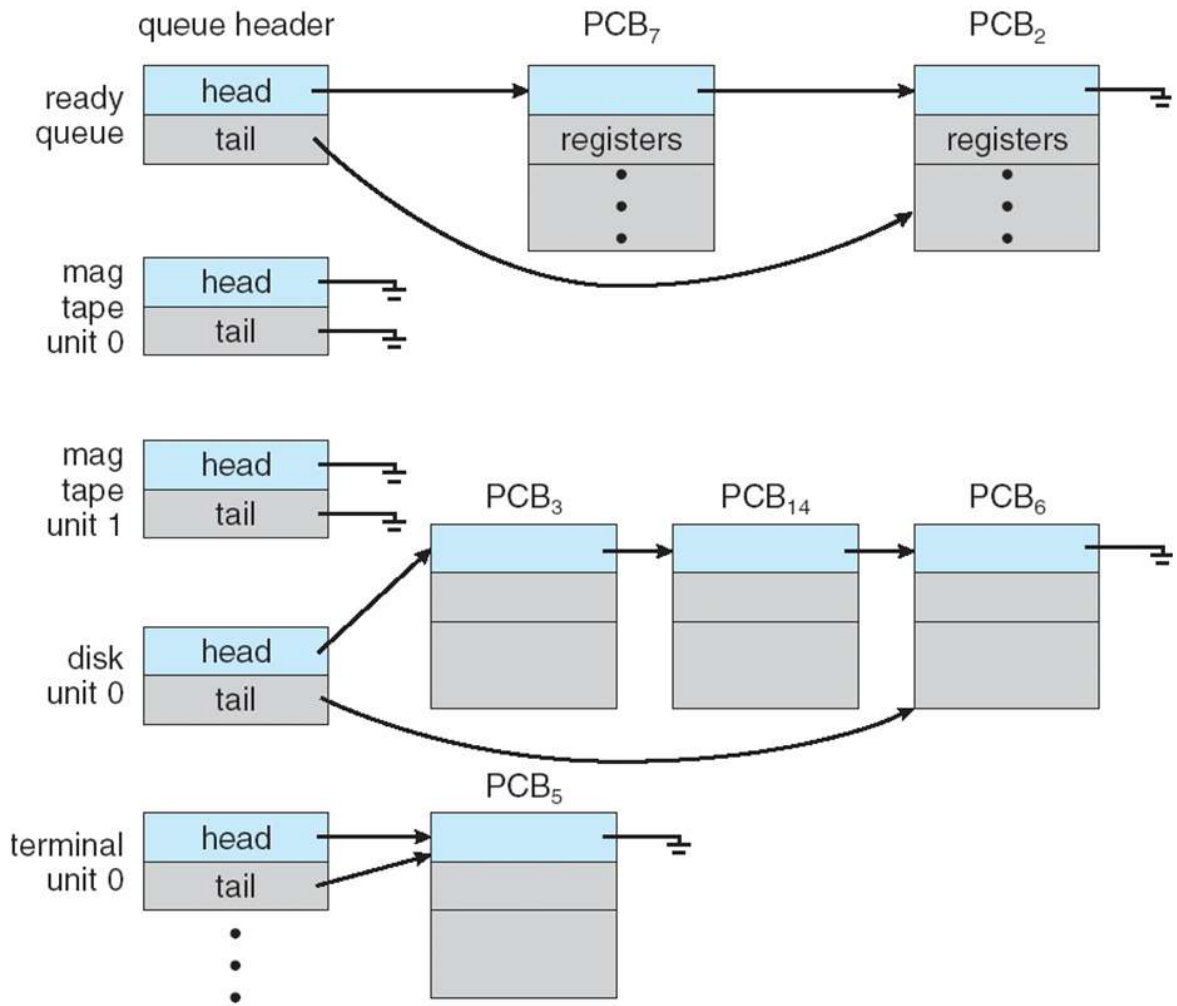
Threads

- So far, process has a single thread of execution
- Consider having multiple program counters per process
  - Multiple locations can execute at once
    - Multiple threads of control -> **threads**
- Must then have storage for thread details, multiple program counters in PCB
- See next chapter

Process Scheduling

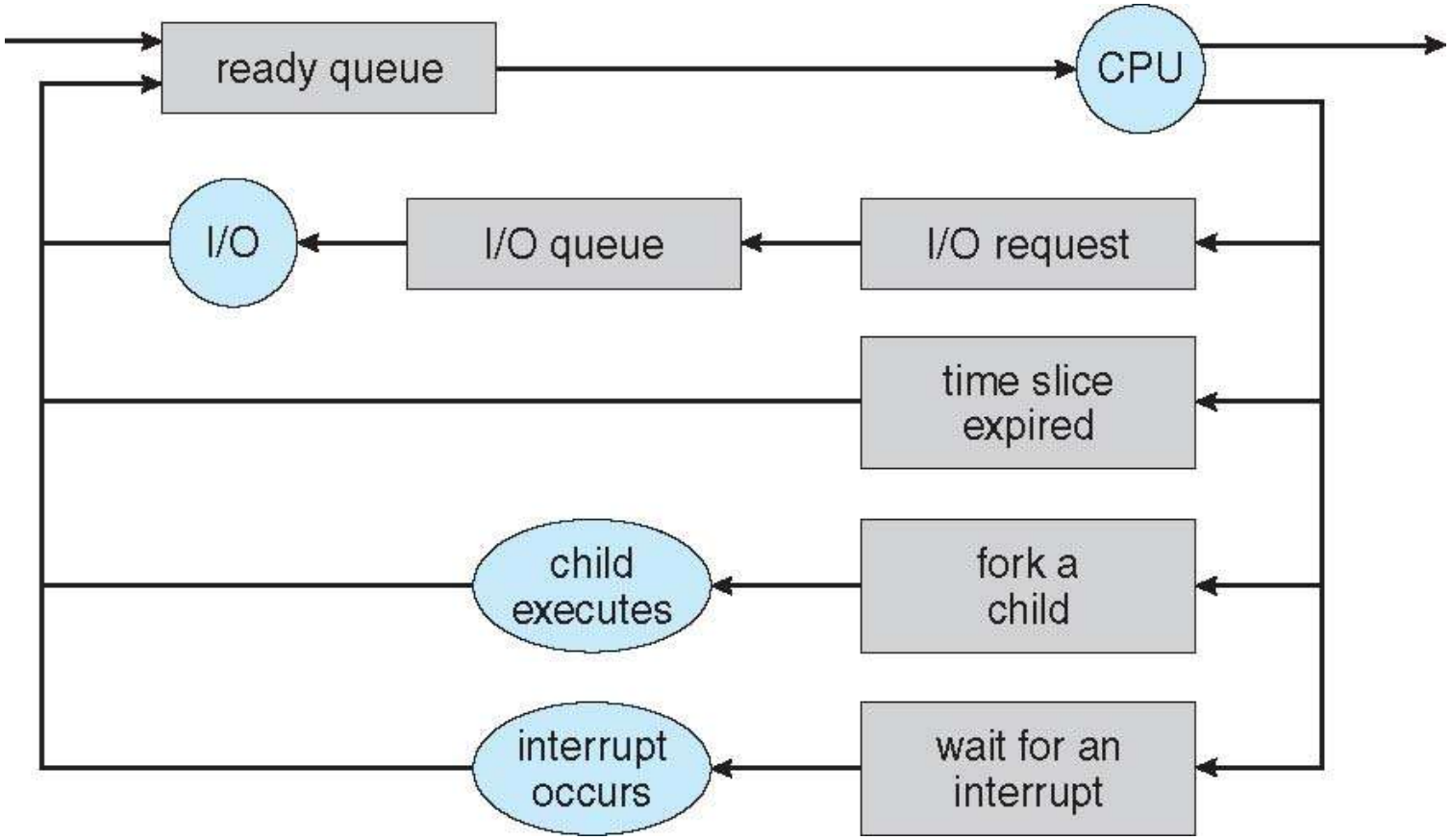
- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU:
  - Single processor
  - Multiple processor
- Maintains **scheduling queues** of processes
  - **Job queue** – set of all processes in the system
  - **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
  - **Device queues** – set of processes waiting for an I/O device
  - Processes migrate among the various queues

Ready Queue And Various I/O Device Queues



Representation of Process Scheduling

Queueing diagram represents queues, resources, flows

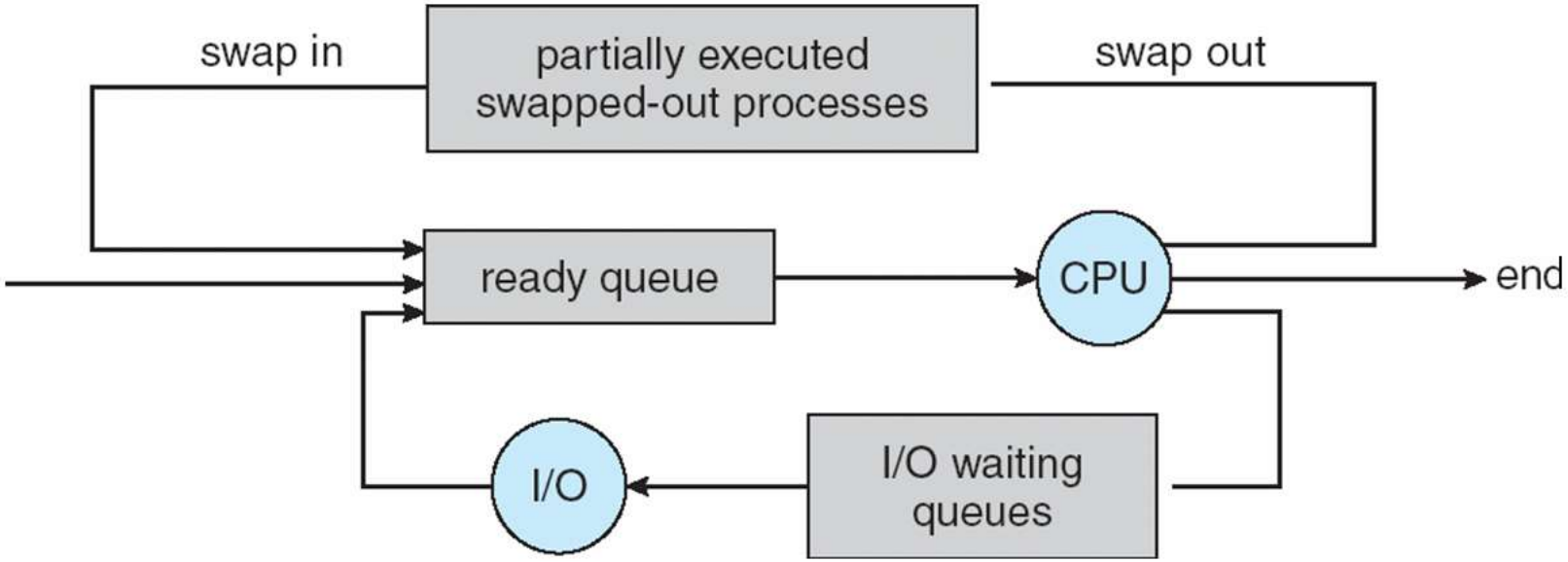


Schedulers

- Selection of process.
- **Short-term scheduler** (or **CPU scheduler**) – selects which process should be executed next and allocates CPU
  - Sometimes the only scheduler in a system
  - Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)
- **Long-term scheduler** (or **job scheduler**) – selects which processes should be brought into the ready queue
  - Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)
  - Long-term scheduler controls the **degree of multiprogramming**
- Processes can be described as either:
  - **I/O-bound process** – spends more time doing I/O than computations, many short CPU bursts
  - **CPU-bound process** – spends more time doing computations; few very long CPU bursts
- Long-term scheduler strives for good ***process mix***
  - May be absent or minimal.

Addition of Medium Term Scheduling

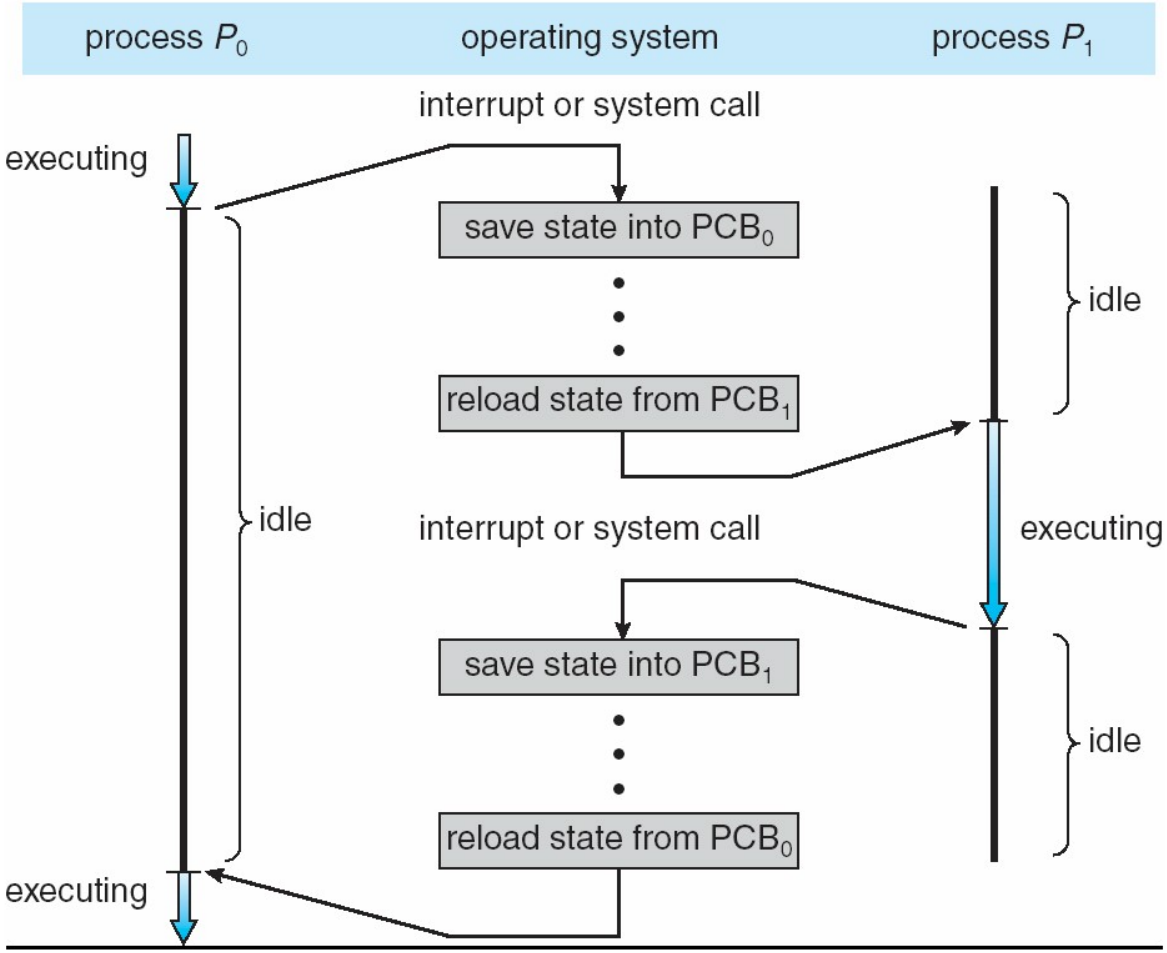
- **Medium-term scheduler** can be added if degree of multiprogramming needs to be decreased
  - Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping (memory management)**



Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB ➔ the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU ➔ multiple contexts loaded at once

CPU Switch From Process to Process

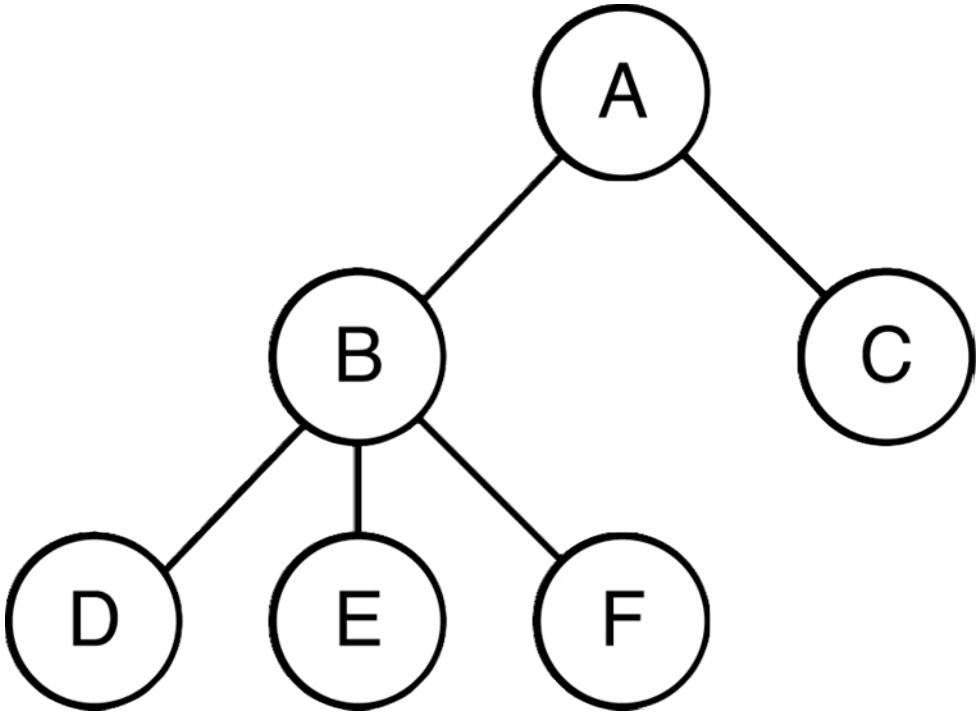


Operations on Processes

- System must provide mechanisms for:
  - process creation
  - process termination

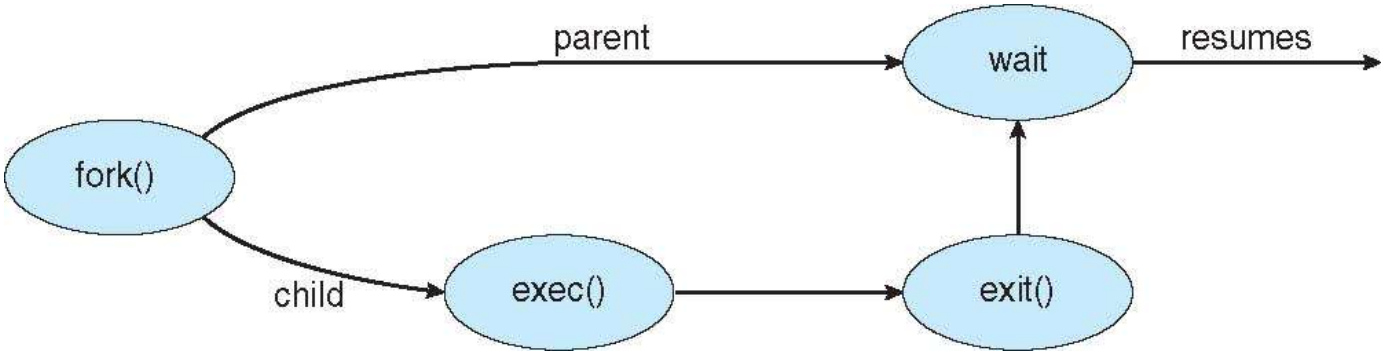
Process Creation

- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier** (**pid**)
- Resource sharing options
  - Parent and children share all resources
  - Children share subset of parent's resources
  - Parent and child share no resources
- Address space
  - Child duplicate of parent (same program and data)
  - Child has a new program loaded into it
- Execution options
  - Parent and children execute concurrently
  - Parent waits until children terminate



- UNIX examples
  - **fork()** system call creates new process
    - The new process (child process) is an exact copy of the calling process (parent process) except the child process has a unique process ID.
  - **exec()** system call used after a **fork()** to replace the process' memory space with a new program
  - **wait()** system call to move itself off the ready queue until the termination of the child
    - The parent process may wait for termination of a child process by using the **wait()**system call.
    - The call returns status information and the pid of the terminated process.

- Both processes (the parent and the child) continue execution at the instruction after the fork():
  - the return code for the fork() is zero for the new (child) process,
  - the return code is positive, if the creation was successful. process identifier (pid) of the child is returned to the parent.
  - the return code for the fork() is negative, if the creation of new (child) process was unsuccessful.



Process Termination

- Process executes last statement and then asks the OS to delete it using the **exit()** system call.
  - Process' resources are deallocated by operating system
  - Returns status data from child to parent (via **wait()**)
    - Entry in the process table must remain until the parent calls wait() - process table contains the exit status.

- Parent may terminate the execution of children processes using the **abort()** system call.
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting and the OS does not allow a child to continue if its parent terminates

- The parent process may wait for termination of a child process by using the **wait()**system call**.**
  If no parent waiting (did not invoke **wait()**) process is a **zombie** – all process transition to this state
    - Once the parent calls wait(), the process identifier of the zombie process and its entry in the process table are released.
  - If parent terminated without invoking **wait** , process is an **orphan**
    - Solution in UNIX – init processs calls wait()

- Some OS do not allow child to exists if its parent has terminated - **cascading termination.**

Examples

Predict the output

```
int main()
{
        fork();
        printf("Hello world!\n");
        return 0;
}
```

- **Including the initial parent process, how many processes are created by the program shown below:**

```
int main()
{
        fork();
        fork();
        fork();
        return 0;
}
```

How many processes are created?

```
int main()
{
        int i;
        for (i = 0; i < 4; i++)
                fork();
        return 0;
}
```

Predict the output

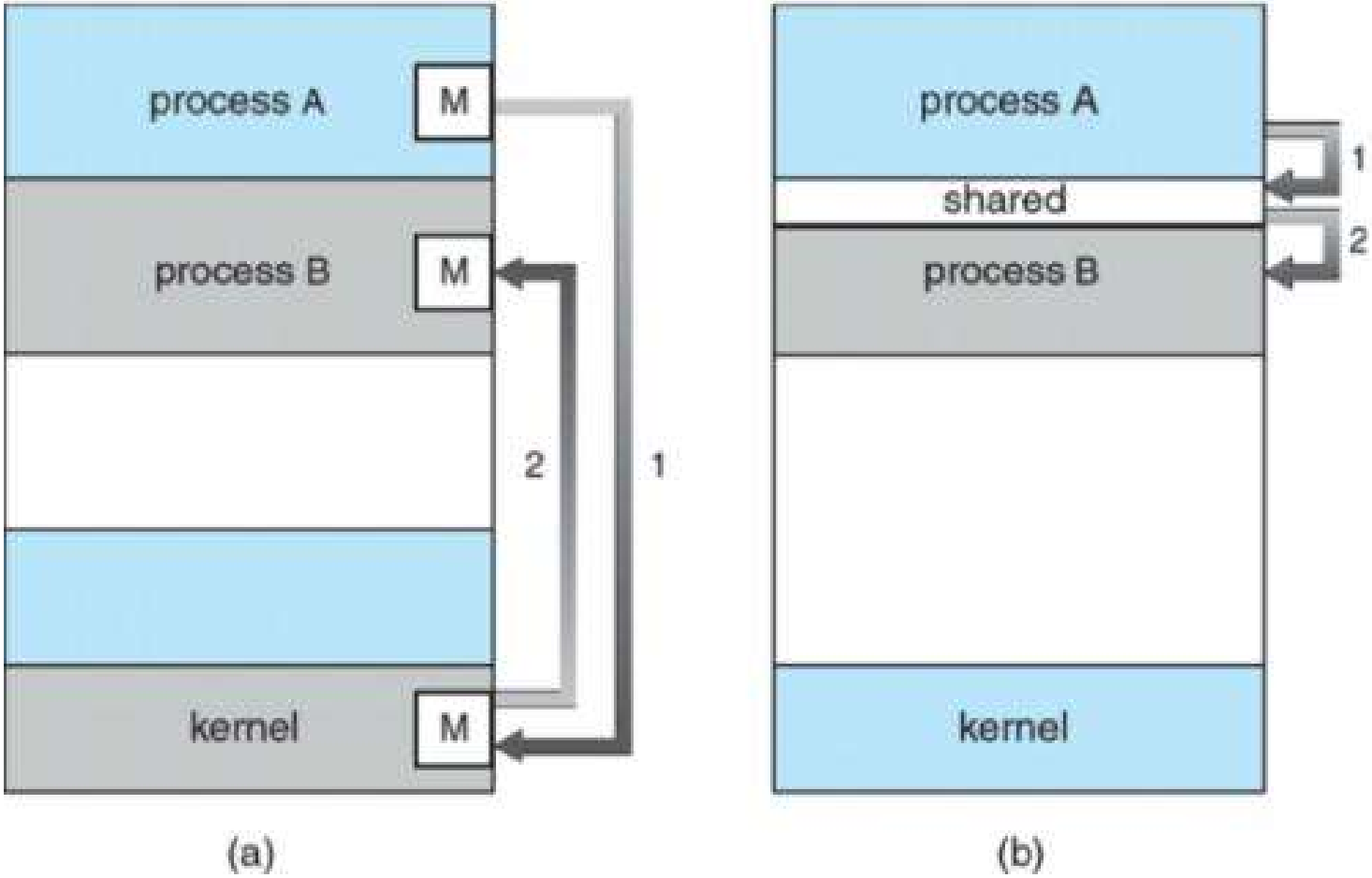
```
void forkexample()
{
        int x = 1;

        if (fork() == 0)
                printf("Child has x = %d\n", ++x);
        else
                printf("Parent has x = %d\n", --x);
}
int main()
{
        forkexample();
        return 0;
}
```

Interprocess Communication

Interprocess Communication

- Processes within a system may be ***independent*** or ***cooperating***
  - ***Independent*** process cannot affect or be affected by the execution of another process
  - ***Cooperating*** process can affect or be affected by the execution of another process
- Reasons for cooperating processes:
  - Information sharing; Computation speedup; Modularity; Convenience
- Cooperating processes need **interprocess communication** (**IPC**)
- Two models of IPC
  - **Shared memory**
  - **Message passing**

Communications Models



- **Shared Memory:**
  - ✓An area of memory shared among the processes that wish to communicate
  - ✓The communication is under the control of the users processes not the OS.
  - ✓Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.

- **Message Passing:**
  - ✓Mechanism for processes to communicate and to synchronize their actions
  - ✓Message system – processes communicate with each other without resorting to shared variables
    - The *message* size is either fixed or variable
  - ✓If processes P and Q wish to communicate, they need to:
    - Establish a communication link between them
    - Exchange messages via send/receive
  - Logical Implementation?

- **Naming**
  - Processes that want to communicate must have a way to refer to each other.
  - **Direct Communication:**
    - Processes must name each other explicitly:
      - Symmetric: **send** (*P*, *message*), **receive**(*Q*, *message*)
      - Asymmetric: **send** (*P*, *message*), **receive**(*id*, *message*)
  - **Indirect Communication:**
    - Messages are directed and received from mailboxes
      - Primitives are defined as: **send**(*A*, *message*), **receive**(*A*, *message*) [‘A’ is mailbox id]
      - Mailbox sharing?

- **Synchronization**
  - Message passing may be either blocking or non-blocking
  - **Blocking** is considered **synchronous - Blocking send, Blocking receive**
  - **Non-blocking** is considered **asynchronous - Non-blocking send**, **Non-blocking receive**
  - Different combinations possible

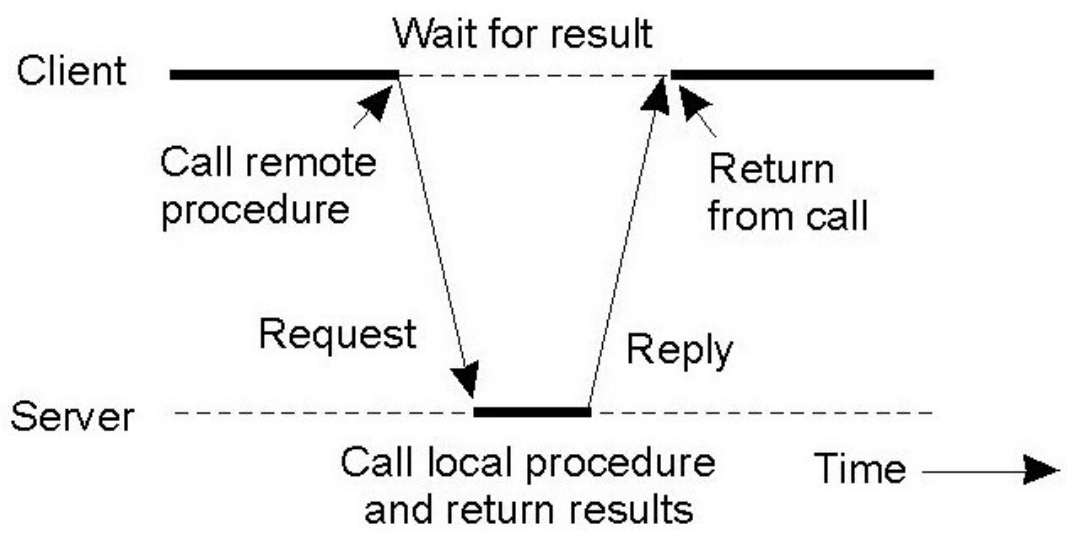
- **Buffering**
  - Queue of messages attached to the link.
  - Implemented in one of three ways
    - ✓ Zero capacity
    - ✓ Bounded capacity
    - ✓ Unbounded capacity

Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)

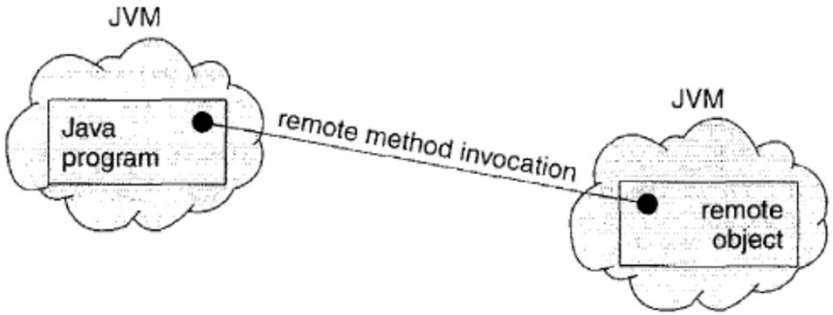
Remote Procedure Call

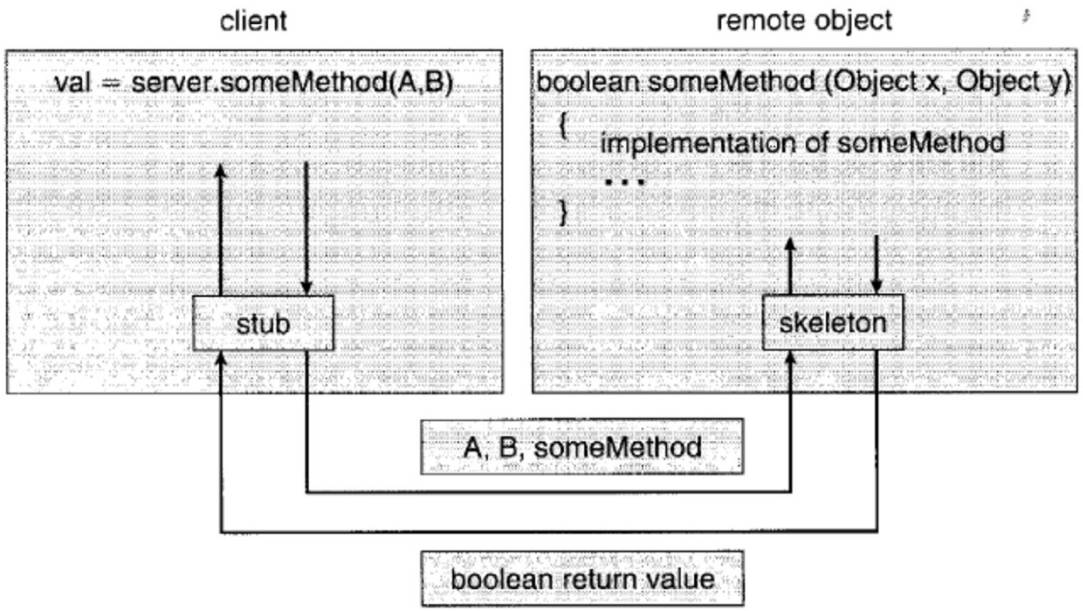
- **Remote Procedure Call (RPC)** is a powerful technique for constructing **distributed, client-server based applications**.
  - extending the conventional local procedure calling, so that the **called procedure need not exist in the same address space as the calling procedure**.
  - The idea behind RPC is to make a remote procedure call look as mush as possible like a local one.



Principle of RPC between a client and server program.

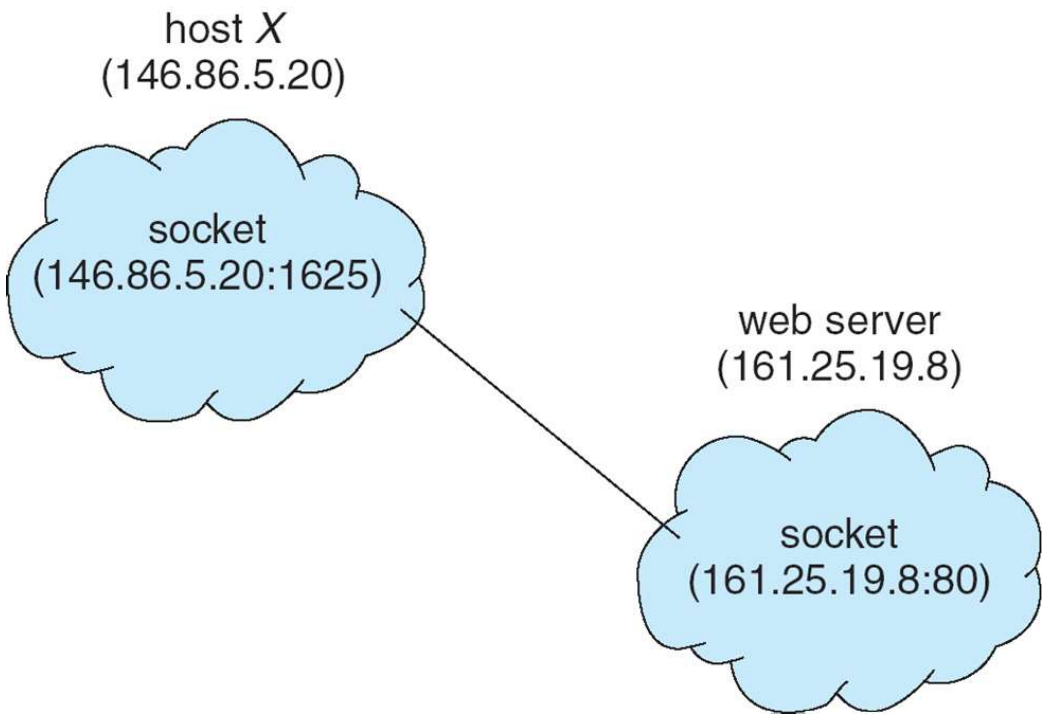
Remote Method Invocation (Java)





Socket

- A **socket** is defined as an endpoint for communication.
- Combination of IP address and **port.**



What is a socket?

- Socket
  - The combination of an IP address and a port number. (RFC 793 ,original TCP specification)
  - Two types
    - Stream socket (connection oriented)
    - Datagram socket (connectionless)

- Socket pair
  - Specifies the two end points that uniquely identifies each connection
  - 4-tuple: (client IP address, client port number, server IP address, server port number)

- Stream Socket:
  - A dedicated point-to-point channel between a client and server.
  - Use TCP (Transmission Control Protocol) for data transmission.
  - Lossless and reliable.
  - Sent and received in the same order.

JAVA Sockets

- **Package java.net**
  - **java.net.Socket**
    - Implements client sockets (also called just “sockets”).
    - Constructor and Methods
      - Socket(String host, int port): Creates a stream socket and connects it to the specified port number on the named host.
      - InputStream getInputStream()
      - OutputStream getOutputStream()
      - close()
  - **java.net.ServerSocket**
    - Implements server sockets.
    - Waits for requests to come in over the network.
    - Performs some operation based on the request.
    - Constructor and Methods
      - ServerSocket(int port)
      - accept(): Listens for a connection to be made to this socket and accepts it. This method blocks until a connection is made.

Implementation in Java

1. Open the Server Socket:

```
ServerSocket server = new ServerSocket( PORT );
```

2. Wait for the Client Request:

```
Socket client = server.accept();  //socket to connect to the client
Socket socket = new Socket(ip,PORT);  //connect to the server – used by client
```

3. Create I/O streams for communicating to the client

```
DataInputStream is = new DataInputStream(client.getInputStream());
DataOutputStream os = new DataOutputStream(client.getOutputStream());
```

4. Perform communication with client

Receive from client: *String line = is.readLine();*

Send to client: *os.writeBytes("Hello \n");*

5. Close socket:

```
client.close();
```

Reference

- "Operating System Concepts", Silberschatz, Galvin, Gagne, 9th Edition, Wiley India.